

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	MANDLA VISWATEJA	Reg. No.:	192311399
Section / Batch:	CSE/2023	Date:	23/07/2026

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Debugging Report: Sorting, Searching & Geometry Algorithms

Error Identification, Root Cause Analysis, and Corrected Implementations

Q1. Debugging Selection Sort Code

Errors Identified

1. Wrong comparison operator (> instead of <).
2. Swaps are performed even when unnecessary.

Root Cause

The algorithm selects the largest element instead of the smallest, producing descending order.

Corrected Code

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_idx = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr
```

Optimization

Swap only if min_idx != i.

Time Complexity

- Best: $O(n^2)$
- Average: $O(n^2)$
- Worst: $O(n^2)$

Space Complexity

$O(1)$

Conclusion: The corrected algorithm properly sorts the array in ascending order.

Q2. Debugging Linear Search Code

Errors Identified

3. Returns -1 when the key is found.
4. Returns index outside the loop.

Root Cause

The function exits immediately without completing the search.

Corrected Code

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
```

```
        return i
    return -1
```

Optimization

Return immediately after finding the key.

Time Complexity

- Best: $O(1)$
- Average: $O(n)$
- Worst: $O(n)$

Space Complexity

$O(1)$

Conclusion: The corrected program searches the entire array correctly.

Q3. Debugging Closest Pair Code

Errors Identified

5. `min_dist = 0`
6. Wrong comparison (`>` instead of `<`)

Root Cause

Since distances are always positive, minimum distance is never updated.

Corrected Code

```
import math

def closest_pair(points):
    min_dist = float('inf')

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            dist = math.sqrt(
                (points[i][0]-points[j][0])**2 +
                (points[i][1]-points[j][1])**2
            )

            if dist < min_dist:
                min_dist = dist

    return min_dist
```

Optimization

Avoid duplicate pair comparisons by starting `j = i + 1`.

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Conclusion: The algorithm now correctly computes the minimum Euclidean distance.

Q4. Debugging Convex Hull Logic

Errors Identified

The orientation formula has the wrong sign.

Root Cause

Incorrect orientation results in interior points being treated as hull points.

Corrected Code

```
def orientation(p, q, r):
    value = ((q[0]-p[0])*(r[1]-p[1]) -
             (q[1]-p[1])*(r[0]-p[0]))

    if value == 0:
        return 0
    elif value > 0:
        return 1
    else:
        return -1
```

Optimization

Stop checking once an invalid edge is detected.

Time Complexity

$O(n^3)$

Space Complexity

$O(1)$

Conclusion: The corrected orientation function correctly identifies convex hull boundaries.

Q5. Debugging Binary Search Code

Errors Identified

7. Wrong boundary updates.
8. Loop condition should check all remaining elements.

Root Cause

The search interval moves in the wrong direction, skipping valid elements.

Corrected Code

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = low + (high - low) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1
```

Optimization

Use $low + (high - low) // 2$ for safe midpoint calculation.

Time Complexity

- Best: $O(1)$
- Average: $O(\log n)$
- Worst: $O(\log n)$

Space Complexity

$O(1)$

Conclusion: *The corrected binary search correctly handles all cases while maintaining logarithmic performance.*